# Optimizing Token Usage in an AI Agent

Hanzhe Wang

## 1 Optimization Techniques

### 1.1 Context compression

A prompt can typically be split into different components: (system) instructions, context (e.g., conversation history or retrieved documents) and the user's question. With the introduction of retrieval-augmented generation (RAG)[1] and multi-turn agents, prompt contexts for large language models (LLMs) have become increasingly large. Yet often parts of the prompts are irrelevant to answering the user's question. This leads to wasted tokens, increased latency, and sometimes reduced accuracy (in long prompts). Different techniques have been developed to reduce the prompt size, thus also the input tokens, while keeping key information. Existing techniques can be broadly categorized into two categories: abstractive methods and extractive methods.

#### 1.1.1 Abstractive context compression

Abstractive methods rewrite large contexts into compact summaries containing only the key information relevant to the user's question. The summarized context is then given to the LLM instead of the original context. CompAct, a representative abstractive approach, rewrites the context using a separate, much cheaper compressor LLM. The compressor LLM iteratively takes a subset of the original context together with the already compressed context and the user question. With the given inputs, it adds relevant information to the compressed context [2]. Through this method, it can significantly compress the context and thus the input tokens for the LLM.

This approach significantly compresses the context for the final LLM and thus the input tokens. Testing has also shown competitive and even superior performance in multi-document contexts compared to other long-context LLMs without compression[2]. However, the compressor LLM still needs to process the original documents and thus, combining the tokens of both LLMs, it actually could be processing more. If looking at cost savings, this approach is very interesting, but latency-wise it could perform worse than without compression. Using a weaker model for compression also comes at a risk of hallucinating or missing facts during compression.

#### 1.1.2 Extractive context compression

Extractive methods do not generate summaries; instead, they remove redundant or irrelevant information from the context. Typically, this is done by scoring the documents and/or the tokens and dropping low-scoring ones. LongLLMLingua does this by using the document's relevance score as a compression budget and dropping predictable tokens (and therefore low information entropy) according to that budget [3].

These methods significantly reduce compression time and token usage compared to abstractive methods. They also do not introduce noise but filter it out, which could improve performance. However, it is hard to tune the exact filter criteria as these lack context awareness.

### 1.2 Token aware reasoning

Besides compressing input tokens, we could also look at output tokens. Reasoning enhancement techniques such as chain-of-thought (CoT) allow LLMs to reason about complex prompts [4]. Often LLMs generate excessive tokens during reasoning, which could be compressed with a token budget in the prompt. Choosing a fitting token budget could effectively reduce LLM output tokens while having minimal impact on performance [5].

In such approaches, it is important to set a correct token budget to effectively compress the output token usage. A token budget too low would have an inverse effect where the LLM does not follow the budget and exceeds it, referred to as "Token Elasticity" [5].

### 1.3 LLM aware retrieval

In RAG systems, retrieval happens first before the LLM is fed a prompt. The retrieved context is the biggest contributor to the input tokens. If the LLM already had the answer stored in its parametric memory, the context would be redundant.

This approach relies on knowing when the LLM has correct information. This can be done by training a distilled classifier model of the LLM predicting whether the knowledge is present. From testing, it follows that the accuracy of such a distilled model enhanced with 32 tokens from the LLM is around 80%. In their case, it saved around 50% of RAG usage; in practice, this is dependent on the use case [6]. Here we have a clear trade-off between performance and accuracy. In heavily redundant RAG cases, this could be an appealing option.

## 2 Preferred approach

### 2.1 Extractive context compression

In RAG-based systems, retrieved documents are the primary driver of input-token inflation. Although abstractive methods have impressive compression capabilities, they increase the latency and potential noise, thus reducing part of the benefits of compression. With an extractive compressor, we only need to drop redundant tokens, making it a highly pragmatic choice for a prototype.

To implement, we could use existing scorers, such as a cross-encoder, to evaluate sentence relevance. These can then be used to score parts (e.g., sentences) of the retrieved context against the user's question. Using the scores, irrelevant parts of the context could be dropped while keeping only the essential information.

### 2.2 Token Aware reasoning

Output tokens take much longer to generate than input tokens take to process. This is why, latency-wise, it is very interesting to look at how we can compress these. Token-aware reasoning is a very pragmatic approach that requires minimal architectural changes, as it is purely prompt-based. This fits perfectly as an initial approach for testing and implementation in our prototype, directly targeting cost and latency.

A pragmatic approach for prototyping is to use an LLM to estimate the prompt budget. The key intuition is that a person may take time to compute an exact answer but can quickly estimate the effort required to solve it [5].

### 2.3 Metrics and evaluation

We should test and compare the prototype with the vanilla RAG baseline and the different compression configurations. Run each of these through the same questions, with the same configuration and models.
We should look at:

- Exact Match (EM) score: This is a binary score indicating whether the LLM answer matches the ground truth exactly. Typically, normalization is applied, removing articles, whitespace, punctuation, and setting letters to lowercase.
- F1 score: The F1 score measures the overlap of the LLM answer with the ground truth. While EM scores very strictly, F1 can be much more lenient and gives points if the answer partially contains the ground truth. The formula: $F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$
- Precision: The fraction of words in the LLM answer that also appear in the ground truth.
- Recall: The fraction of words in the ground truth that also appear in the LLM answer.
- End-to-end inference latency: in seconds.
- Input token usage.
- Output token usage.

A competitive compressing technique should keep the accuracy (EM & F1 scores) as high as possible, while reducing latency and token usage.

# 3 Experiment proposal

Here, we investigate how effective the preferred approaches, as described in section 2, are in a simple RAG setup. For this setup, we used "openai/gpt-oss-120b", a lightweight model accessible via groq; it is free to use and has generous limits [7]. For the embedding model, "sentence-transformers/all-MiniLM-L6-v2" is used, and for the cross-encoder, "cross-encoder/ms-marco-MiniLM-L-6-v2" is used. Both are lightweight models accessible via HuggingFace [8] [9]. The implementation is done in Python using LangChain.

For testing, we used the hotpot_qa dataset as it requires multi-hop reasoning while the answers remain simple. The simple QA answers lean well into metrics such as Exact Match and F1. We also look at the end-to-end latency, the input-token count, and the output-token count. Using these metrics, we tested four different methods: vanilla (a baseline naive RAG model), context compressed (RAG with sentence-level context compression), reason budget (RAG with reason budgeting in prompt), and fully compressed (this combines the two compression methods).

| Method | EM | F1 | Latency | Input | Output | Total | Compression | Speed |
|---|---|---|---|---|---|---|---|---|
| vanilla | **66.00%** | **77.23%** | 3.841 | 562.2 | 124.0 | 34307 | 1.00 | 1.00 |
| reason budget (token) | 56.00% | 69.19% | 5.143 | 569.2 | **110.5** | 33985 | 0.99 | 1.34 |
| context compressed | 64.00% | 74.00% | **2.974** | **276.6** | 123.0 | **19980** | **0.58** | **0.77** |
| fully compressed (token) | 52.00% | 66.01% | 3.141 | 283.6 | 131.8 | 20774 | 0.61 | 0.82 |

**Table 1** Performance of the different methods on 50 questions from hotpot_qa. EM and F1 are average scores per question. Latency is the average end-to-end latency per question in seconds. Input and output are the average token count per question. Total is the total count of tokens used for the entire 50 questions. Compression is the ratio (method total tokens / vanilla total tokens). Speed is the ratio (method latency / vanilla latency).

For context compression, we first split the retrieved documents into sentences. Afterward, we scored the relevance of the sentences using the cross-encoder on the (sentence, question) pairs. We then took the top 4 sentences for each question as its context instead of all the retrieved documents. Table 1 shows how context compression delivers an impressive token compression of 58%. This translates into a latency reduction of 33%. While the accuracy of the answers drops by 2 points on EM and 3.23 points on the F1 score, this illustrates extractive context compression's ability to effectively reduce token usage while having minimal impact on accuracy.

For the sake of a proof of concept and the scope of this experiment, the budgeting for each question is the same and chosen beforehand. We used a budget of 84 tokens in the prompt. As shown by Table 1, the LLM fails to adhere to this limit. Although it does succeed in reducing the output tokens, it comes at a cost of increased input tokens and an increased latency of 34%. This illustrates how LLMs have a hard time reasoning about their output tokens and how it affects their inference time. Above all, it also performs worse on accuracy, scoring 10 points lower on EM and 8.04 points lower on the F1 score. This suggests alternative implementations or approaches should be chosen instead.

The fully compressed model has the worst EM and F1 scores as it inherits both performance drops. While compression and latency wise it performs worse than the context compressed model, due to the negative impact of the token budget.

## References


[1] Patrick S. H. Lewis, A.P.F.P.V.K.N.G.H.K.M.L.W.-t.Y.T.R.S.R. Ethan Perez, Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html (2020)

[2] Chanwoong Yoon, H.H.M.J.J.K. Taewhoo Lee: COMPACT: Compressing Retrieved Documents Actively for Question Answering. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 21424–21439 November 12-16, 2024 ©2024 Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-main.1194/ (2024)

[3] Huiqiang Jiang, X.L. Qianhui Wu: LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. arXiv preprint arXiv:2310.06839 https://arxiv.org/html/2310.06839v2 (2024)

[4] Jason Wei, D.S.M.B.B.I.F.X.E.C.Q.L.D.Z. Xuezhi Wang: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903v6 https://arxiv.org/abs/2201.11903 (2022)

[5] Tingxu Han, C.F.S.Z.S.M.Z.C. Zhenting Wang: Token-Budget-Aware LLM Reasoning. arXiv preprint arXiv:2412.18547 https://arxiv.org/abs/2412.18547 (2026)

[6] Déjean, H.: Let your LLM generate a few tokens and you will reduce the need for retrieval. arXiv preprint arXiv:2412.11536 https://arxiv.org/abs/2412.11536 (2024)

[7] Groq: GPT OSS 120B. https://console.groq.com/docs/model/openai/gpt-oss-120b

[8] HuggingFace: all-MiniLM-L6-v2. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[9] HuggingFace: Cross-Encoder for MS Marco. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2